

Estructura de carpetas de un backend en formato MVC

En general los proyectos pueden variar en carpetas, pero básicamente, la **estructura típica de carpetas en un proyecto MVC (Model–View–Controller)** sigue un patrón muy claro: separar **lógica, datos y presentación**.

Las fuentes consultadas describen esta organización en proyectos MVC clásicos:

- **Controllers:** clases que manejan las solicitudes del usuario.
- **Models:** clases que representan los datos y la lógica de negocio.
- **Views:** archivos HTML o plantillas que muestran la interfaz.

Estructura esencial de carpetas en un proyecto MVC

/mi-proyecto

|— /controllers

|— /models

|— /views

|— /public

| |— css/

| |— js/

| └─ img/

|— /routes

|— /config

└─ app.js o server.js

En las carpetas podríamos colocar;

1) /controllers

Contiene los controladores, que reciben las peticiones y deciden qué hacer.

Ejemplo: UserController.js, ProductController.js.

Los controladores suelen terminar en *Controller*, como indica la documentación MVC.

2) /models

Aquí van los modelos de datos: clases, esquemas o estructuras que representan la información.

Ejemplo: User.js, Product.js.

Los modelos encapsulan reglas de negocio y validaciones

3) /views

Contiene las vistas: HTML, plantillas EJS, Handlebars, Pug, etc.
Cada controlador suele tener su propia subcarpeta de vistas.

Ejemplo:

/views

```
├— user/  
├— product/  
└— shared/
```

Esto coincide con la estructura MVC tradicional donde cada controlador tiene su carpeta de vistas.

4) /public

Carpeta para archivos estáticos:

- CSS
- JavaScript del frontend
- Imágenes
- Iconos

Esto es equivalente a la carpeta *Content* y *Scripts* en proyectos MVC clásicos.

5) /routes

No siempre aparece en MVC clásico, pero es estándar en proyectos JavaScript (Node.js + Express).

Aquí se definen las rutas que apuntan a los controladores.

6) /config

Archivos de configuración:

- Conexión a base de datos
- Variables de entorno
- Configuración de rutas (similar a *App_Start* en MVC).

7) Archivo principal

app.js o server.js

Es el punto de entrada del servidor.

Estructura MVC típica en JavaScript (Node + Express)

/mi-proyecto

```
|— app.js
|— /controllers
|— /models
|— /views
|— /routes
|— /public
|   |— css/
|   |— js/
|   └─ img/
└─ /config
```

Veamos un ejemplo:

/mi-proyecto-mvc

```
|— app.js
|— package.json
|— .env
|— .gitignore
|
|— /config
|   |— db.js
|   └─ server.js
|
|— /routes
|   |— index.routes.js
|   |— user.routes.js
|   └─ product.routes.js
|
|— /controllers
```

```
|   ├── HomeController.js
|   ├── UserController.js
|   └── ProductController.js
|
|   ├── /models
|   |   ├── User.js
|   |   └── Product.js
|   |
|   |   ├── /views
|   |   |   ├── layout.ejs
|   |   |   ├── home.ejs
|   |   |   ├── /user
|   |   |   |   ├── login.ejs
|   |   |   |   └── profile.ejs
|   |   |   └── /product
|   |   |       ├── list.ejs
|   |   |       └── detail.ejs
|   |   |
|   |   ├── /public
|   |   |   ├── /css
|   |   |   |   └── styles.css
|   |   |   ├── /js
|   |   |   |   └── main.js
|   |   |   └── /img
|   |   |       └── logo.png
|   |   |
|   |   └── /middlewares
|   |       ├── auth.js
|   |       └── logger.js
```

Estructura completa de un proyecto MVC en JavaScript

/mi-proyecto-mvc

```
|— app.js
|— package.json
|— .env
|— .gitignore
|
|— /config
|   |— db.js
|   |— server.js
|
|— /routes
|   |— index.routes.js
|   |— user.routes.js
|   |— product.routes.js
|
|— /controllers
|   |— HomeController.js
|   |— UserController.js
|   |— ProductController.js
|
|— /models
|   |— User.js
|   |— Product.js
|
|— /views
|   |— layout.ejs
|   |— home.ejs
|   |— /user
|       |— login.ejs
```

```
| | └─ profile.ejs
| └─ /product
|   └─ list.ejs
|   └─ detail.ejs
|
└─ /public
    └─ /css
        └─ styles.css
    └─ /js
        └─ main.js
    └─ /img
        └─ logo.png
    └─ /middlewares
        └─ auth.js
        └─ logger.js
```

Explicación de cada carpeta

1) /config

Contiene configuraciones globales:

- db.js → conexión a la base de datos
- server.js → configuración del servidor Express

2) /routes

Define las rutas del sistema, separadas por entidad:

- user.routes.js
- product.routes.js
- index.routes.js (rutas generales)

Cada ruta llama a un controller.

3) /controllers

Controladores que procesan la lógica de cada solicitud:

- reciben la petición
- llaman al modelo
- devuelven la vista o JSON

Ejemplo: UserController.js.

4) /models

Modelos de datos:

- Representan entidades
- Contienen validaciones y reglas de negocio

Ejemplo: User.js.

5) /views

Vistas renderizadas por el servidor:

- Plantillas EJS, Pug o Handlebars
 - Subcarpetas por módulo
-

6) /public

Archivos estáticos:

- CSS
 - JS del frontend
 - Imágenes
-

7) /middlewares

Funciones que se ejecutan antes de los controladores:

- autenticación
 - logs
 - validaciones
-

8) Archivos raíz

- app.js → punto de entrada del proyecto
-

- package.json → dependencias y scripts
- .env → variables de entorno
- .gitignore → exclusiones de Git

Mario Bressano

Mayo 2026